



PENSAMENTO COMPUTACIONAL • AULA 14 DE 15

# Complexidade de Algoritmos

*Da função de custo ao Big O, com experimentos em Julia + Pluto — roteiro visual para aula ou minicurso.*

# Objetivos de aprendizagem

- 1 Construir a função de custo  $T(n)$  a partir da contagem de passos.
- 2 Interpretar sequência, decisão, repetição e recursão como contribuições distintas ao custo.
- 3 Diferenciar eficiência prática, escalabilidade e classificação assintótica.
- 4 Usar Julia + Pluto para testar hipóteses com gráficos e sliders.

## A pergunta central

- Um algoritmo que funciona para 100 entradas ainda será viável para 100 mil?
- A análise de complexidade não pergunta apenas “funciona?”. Ela pergunta “como o trabalho cresce?”.
- Essa pergunta antecede o tempo em segundos, porque segundos mudam com a máquina; passos pertencem ao algoritmo.

**passos** ↑↑    **em função de**    **n** ↑

## Passos, não segundos

### Tempo medido

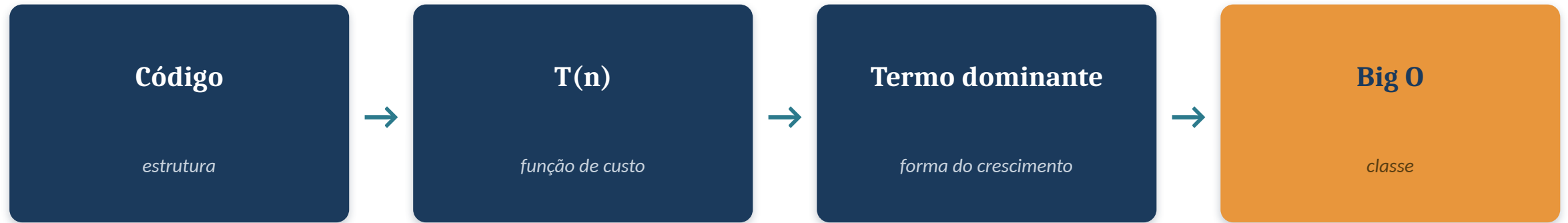
- Depende de processador, cache, compilador, linguagem, SO, I/O e concorrência.

### Passos lógicos

- Pertencem ao algoritmo e permitem comparar soluções antes de executar.

*Complexidade começa na contagem de ações fundamentais.*

## Do código ao Big O



*A ordem assintótica é o último passo. Ela não substitui a função de custo.*

## Ação fundamental

- Antes de contar, é preciso definir o que será contado.
- comparação  $\rightarrow v[i] > 0$
- soma  $\rightarrow s += v[i]$
- chamada  $\rightarrow \text{fib}(n-1)$
- A escolha precisa ser coerente durante toda a análise.

## Sequências calibram o crescimento

- As três ações dentro do laço geram uma constante multiplicativa.
- $T(n) = 3n$
- No Big O, isso vira  $O(n)$ , mas a constante representa trabalho real.

```
for i in 1:n
    a += 1
    b += 1
    c += 1
end
```

$$T(n) = an + b$$

## Constante aditiva e multiplicativa

- Antes do laço: custo fixo.
- Dentro do laço: custo por elemento.
- Depois do laço: custo fixo.
- Exemplo aproximado:  $T(n) = 2n + 4$

```
x = 0
y = 0
z = 0

for i in 1:n
    soma += v[i]
    cont += 1
end

media = soma / cont
```

# Condicionais ajustam caminhos

- Mesmo com dois caminhos, apenas um é executado.
- O custo costuma permanecer constante:  $O(1)$ .
- Mas condicionais importam para melhor caso, pior caso e caso médio.

```
if x > 0
    soma += 1
else
    soma -= 1
end
```

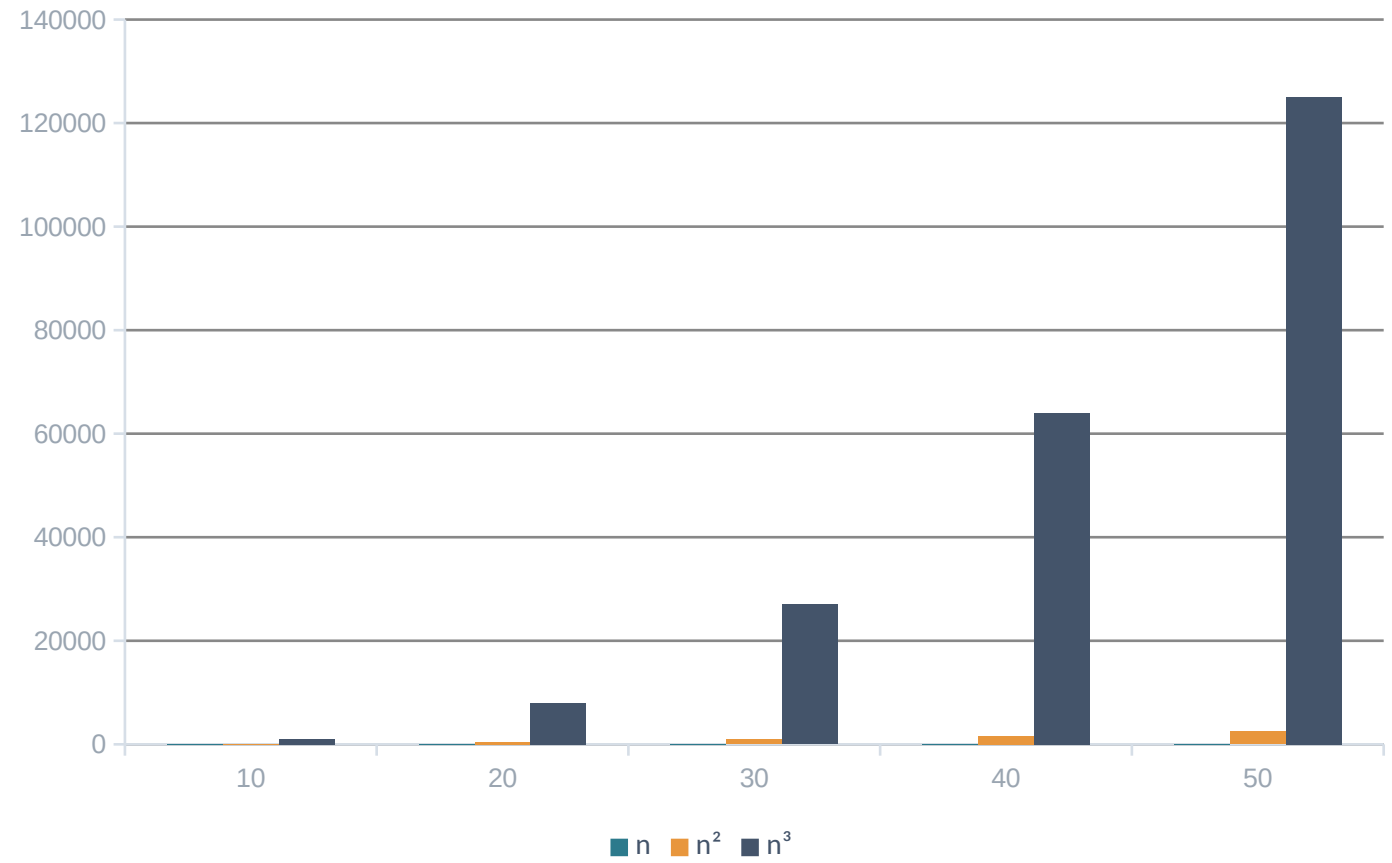
## O IF dentro do laço

- O IF é constante, mas está sendo executado  $n$  vezes.
- $T(n) \approx c \cdot n$
- A pergunta prática é: quantas vezes esta decisão será tomada?

```
for i in 1:n
    if v[i] > 0
        cont += 1
    end
end
```

## Laços multiplicam trabalho

- Cada laço aninhado introduz uma nova dimensão de crescimento.
- 1 laço  $\rightarrow O(n)$
- 2 laços  $\rightarrow O(n^2)$
- 3 laços  $\rightarrow O(n^3)$



## EXEMPLO

# Laço simples: crescimento linear

- Se há uma ação principal por elemento:  $T(n) = n$
- Custo marginal:  $T(n) - T(n-1) = 1$

```
function soma(v)
  s = 0
  for x in v
    s += x
  end
  return s
end
```

**function soma(v) — qual é a resultante para um vetor genérico?**

```
function soma(v)
  s = 0
  for x in v
    s += x
  end
  return s
end
```

**s** → parte de 0

**para cada x em v** → s acumula  $s + x$  (uma soma por elemento — não há valor concreto de v neste slide, então o traço é estrutural)

**soma(v)** → retorna  $s =$  soma de todos os elementos de v, ao final do laço

## Laço duplo: crescimento quadrático

- O laço interno executa  $n$  vezes para cada valor de  $i$ .
- $n \cdot n = n^2$
- Custo marginal aproximado cresce com  $n$ .

```
for i in 1:n
    for j in 1:n
        cont += 1
    end
end
```

## Multiplicação clássica de matrizes

- Há  $n^2$  posições em C.
- Para cada posição, somamos  $n$  produtos.
- $T(n) \approx n^2 \cdot n = n^3$
- Este exemplo mostra como a estrutura do problema impõe crescimento.

```
for i in 1:n
  for j in 1:n
    C[i,j] = 0
    for k in 1:n
      C[i,j] += A[i,k]*B[k,j]
    end
  end
end
```

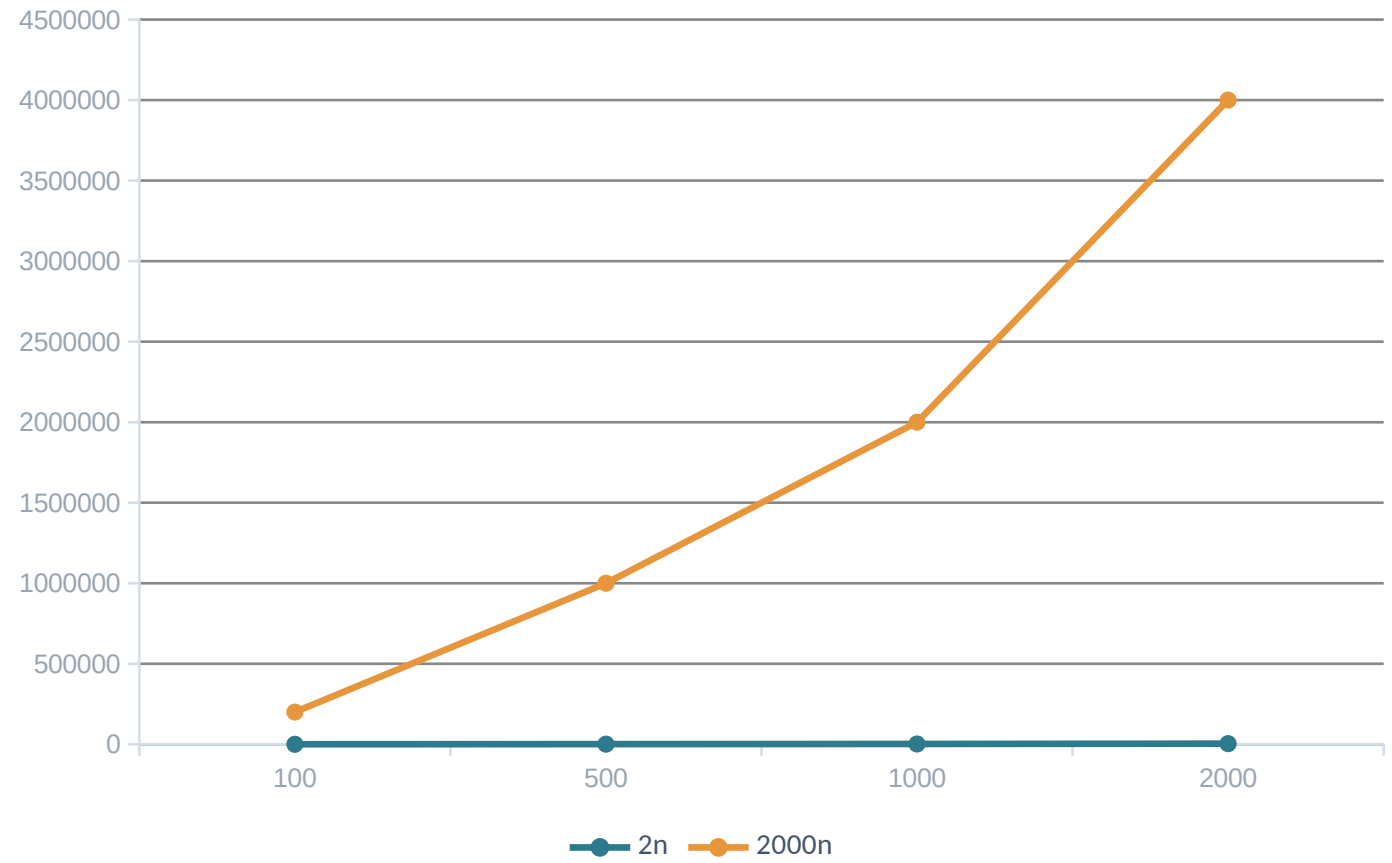
## Big O e o papel de $n_0$

- O  $n_0$  admite que o início da curva pode ser irregular.
- Big O não nega as constantes. Ele escolhe olhar para a região assintótica.

$f(n) = O(g(n))$  quando existem  $c > 0$  e  $n_0$  tais que  $f(n) \leq c \cdot g(n)$  para todo  $n \geq n_0$

## Constantes escondidas ainda importam

- Ambas são  $O(n)$ .
- Mas a constante multiplicativa muda o custo concreto.
- Big O avalia escalabilidade;  $T(n)$  preserva eficiência aproximada.



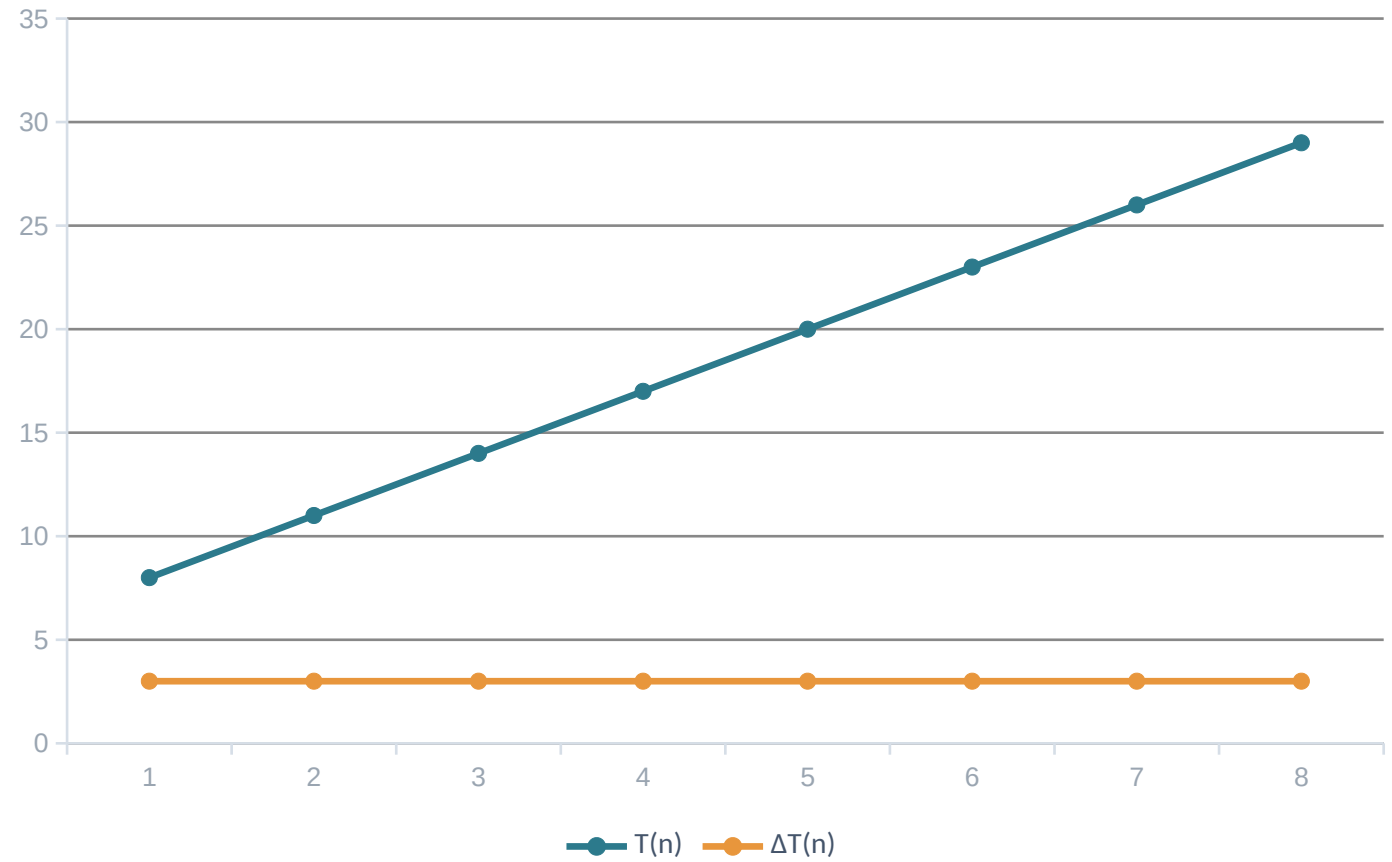
## Coeficiente angular e custo marginal

- Para  $T(n) = an + b$ , o coeficiente  $a$  é a inclinação da reta de custo.
- Interpretação: quanto custa adicionar mais um elemento à entrada.
- Essa ideia aproxima análise de algoritmos e cálculo discreto.

**Derivada discreta:  $\Delta T(n) = T(n) - T(n-1)$**

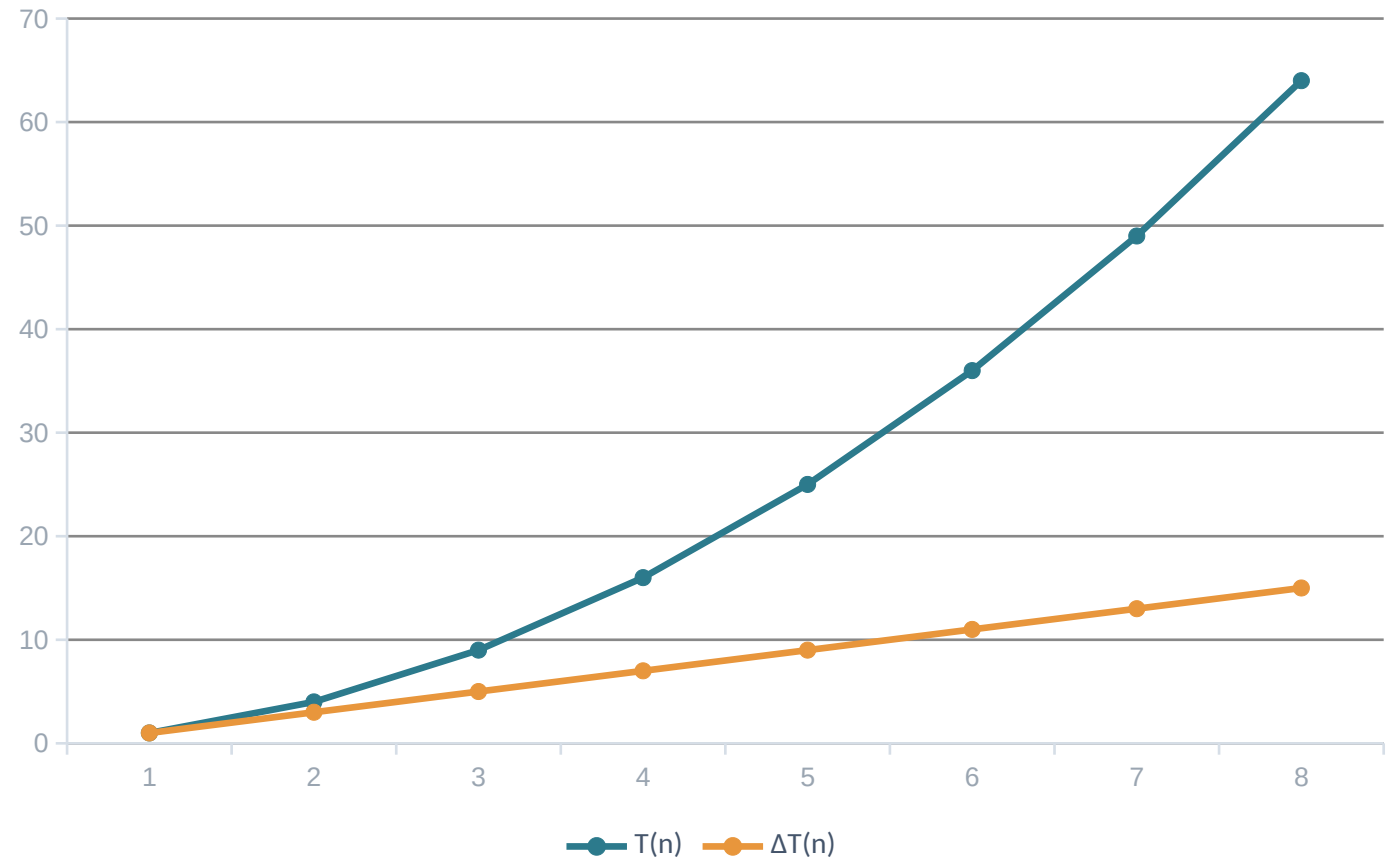
## Custo local estável

- Cada novo elemento acrescenta praticamente o mesmo bloco de trabalho.
- Exemplo: percorrer um vetor uma única vez.
- $T(n) = 3n + 5$  e  $\Delta T(n) = 3$



## Custo local crescente

- O trabalho adicional por novo elemento cresce com  $n$ .
- Por isso algoritmos quadráticos ficam rapidamente caros.
- $T(n) = n^2$  e  $\Delta T(n) = 2n - 1$



## Fibonacci como função

- A definição é curta, mas a implementação recursiva ingênua gera uma árvore de chamadas.
- O problema não é “usar recursão”. O problema é recalcular os mesmos subproblemas.

$$F(0) = 0 \quad F(1) = 1 \quad F(n) = F(n-1) + F(n-2)$$

## Fibonacci recursivo ingênuo

- Há poucas linhas de código, mas muitas chamadas repetidas.
- Linhas de código não medem complexidade.
- A estrutura de chamadas é o objeto da análise.

```
function fib(n)
    if n <= 1
        return n
    end
    return fib(n-1) + fib(n-2)
end
```

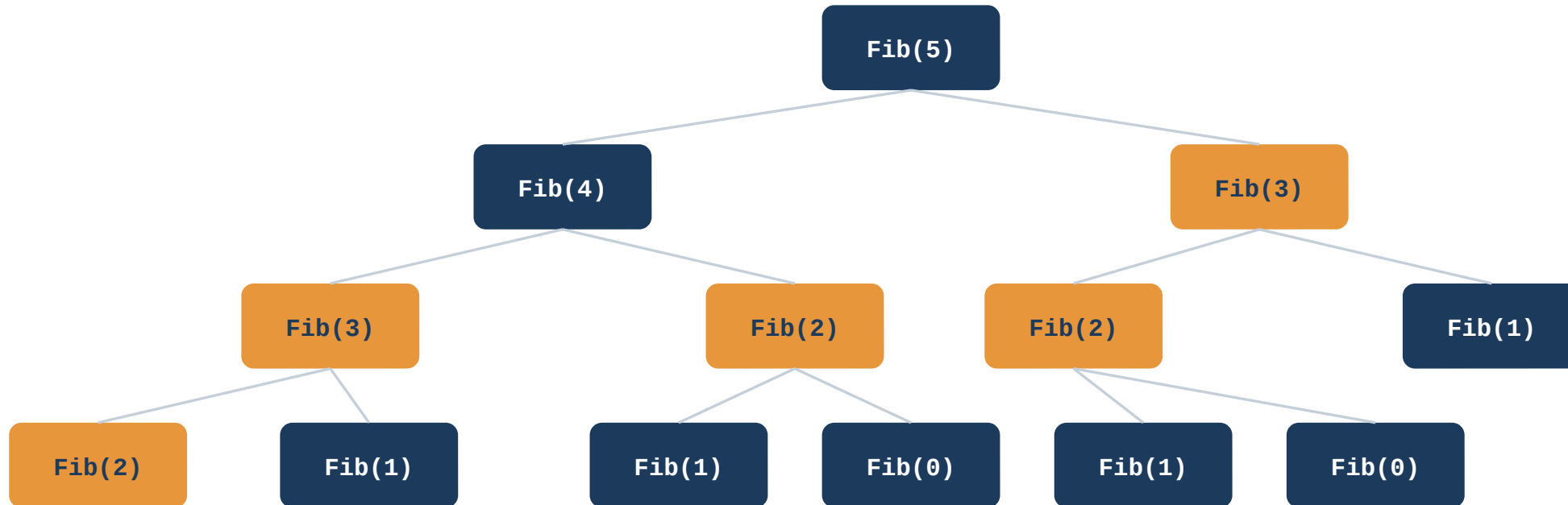
## function fib(n) — árvore de chamadas para fib(5)

```
function fib(n)
  if n <= 1
    return n
  end
  return fib(n-1) + fib(n-2)
end

# chamada: fib(5)
```

```
fib(5) → = fib(4) + fib(3)
fib(4) → = fib(3) + fib(2)
fib(3) (chamada 2×) → = fib(2) + fib(1), em cada uma das duas vezes
fib(2) (chamada 3×) → = fib(1) + fib(0), em cada uma das três vezes
fib(1) (caso-base, chamado 5×) → retorna 1 diretamente, sem nova chamada
fib(0) (caso-base, chamado 3×) → retorna 0 diretamente, sem nova chamada
total de chamadas à função fib → 15 (1×fib(5) + 1×fib(4) + 2×fib(3) + 3×fib(2) + 5×fib(1) + 3×fib(0))
fib(5) → 5 (resultado final, depois de somar toda a árvore)
```

## Árvore de chamadas



*Fib(3) e Fib(2) aparecem mais de uma vez. Isso é recomputação.*

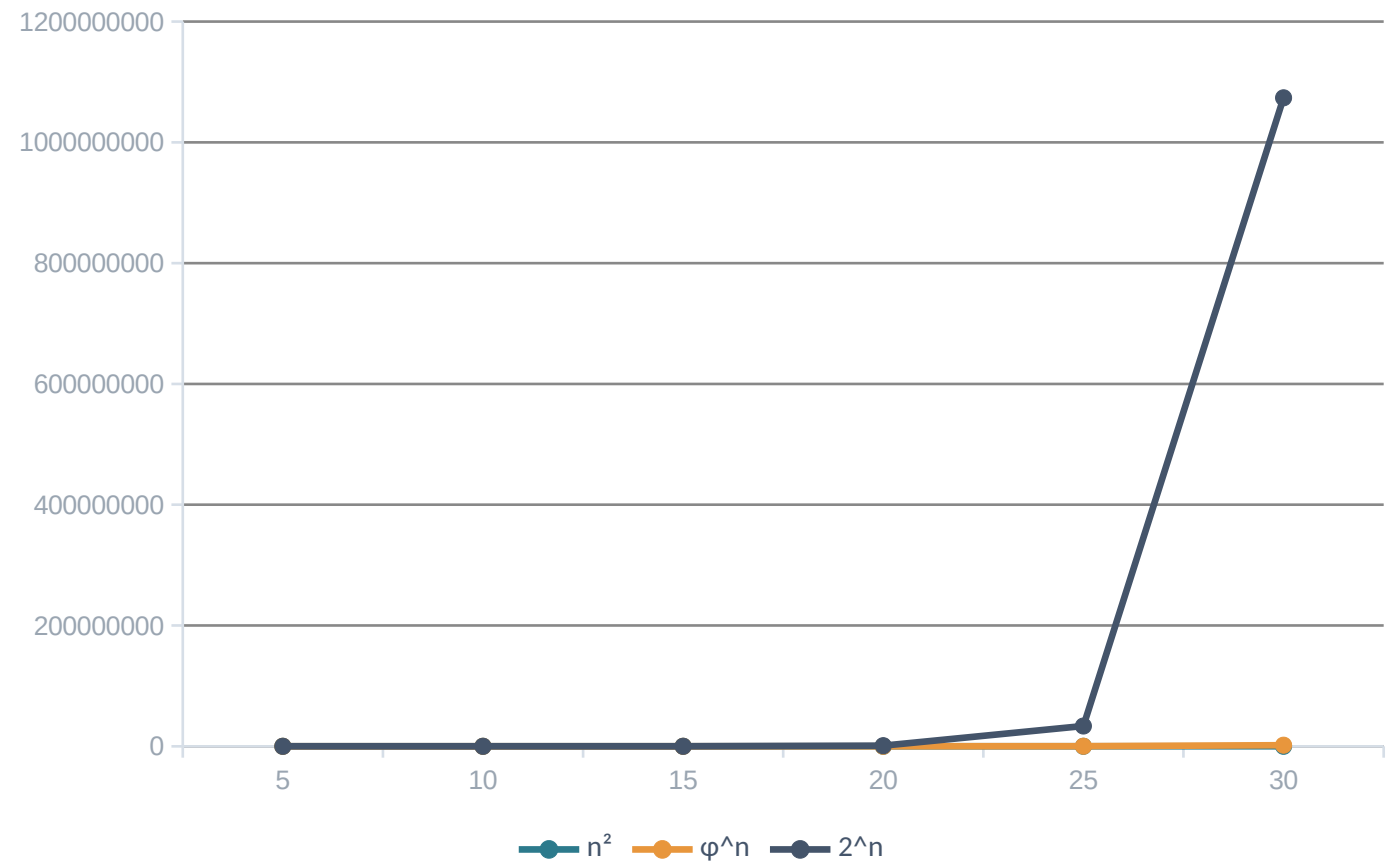
## Custo do Fibonacci ingênuo

- Crescimento aproximado:  $T(n) = T(n-1) + T(n-2) + 1$
- A soma das duas chamadas gera uma expansão em árvore.
- $\varphi = (1+\sqrt{5})/2 \approx 1,618$

$$T(n) = T(n-1) + T(n-2) + 1 \quad \rightarrow \quad O(\varphi^n)$$

## Polinomial versus exponencial

- Em  $n^2$ , a variável está na base.
- Em  $\varphi^n$ , a variável está no expoente.
- Essa diferença explica a explosão.



## Recursão não é sinônimo de explosão

### Linear

- $T(n) = T(n-1) + 1$
- $O(n)$

### Logarítmica

- $T(n) = T(n/2) + 1$
- $O(\log n)$

### Com sobreposição

- $T(n) = T(n-1) + T(n-2) + 1$
- $O(\varphi^n)$

*A estrutura dos subproblemas determina o crescimento.*

# Programação dinâmica

## Sem memória

- O mesmo subproblema é resolvido várias vezes.

## Com memoização

- Cada estado  $F(i)$  é calculado uma única vez e armazenado.
- Custo:  $O(n)$

## Fibonacci iterativo

- O algoritmo percorre os estados de 0 até n.
- Tempo:  $O(n)$
- Memória:  $O(1)$
- Não há recomputação.

```
function fib_iter(n)
    a, b = 0, 1
    for i in 2:n
        a, b = b, a + b
    end
    return n == 0 ? 0 : b
end
```

## function fib\_iter(n) — passo a passo para fib\_iter(5)

```
function fib_iter(n)
    a, b = 0, 1
    for i in 2:n
        a, b = b, a + b
    end
    return n == 0 ? 0 : b
end

# chamada: fib_iter(5)
```

**a, b** → começam em 0, 1

**i = 2** → a, b = 1, 0+1 → a=1, b=1

**i = 3** → a, b = 1, 1+1 → a=1, b=2

**i = 4** → a, b = 2, 1+2 → a=2, b=3

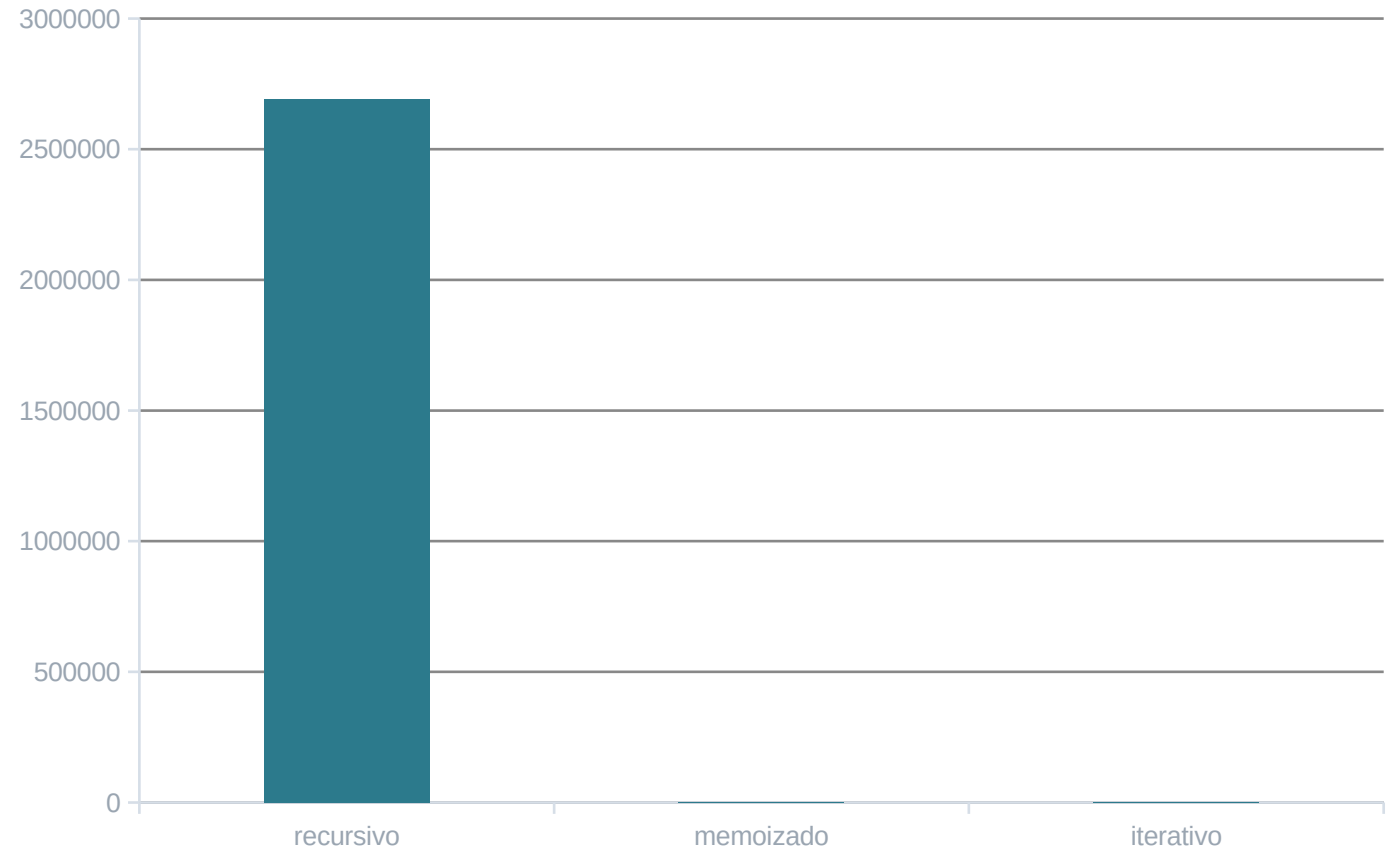
**i = 5** → a, b = 3, 2+3 → a=3, b=5

**fib\_iter(5)** 5 (n ≠ 0, então retorna b)

N = 30

## Comparando estratégias para Fibonacci

- A programação dinâmica não muda a definição de Fibonacci.
- Ela muda a organização do cálculo.
- Resultado: de exponencial para linear.



## Julia + Pluto

- Pluto permite transformar complexidade em experimento visual.
- Sliders → alteram n em tempo real.
- Gráficos → mostram crescimento.
- Código → gera a própria evidência.

## Estrutura do notebook Pluto

- O notebook acompanha a apresentação e contém quatro experimentos.
- 1. Contagem de operações em laços
- 2. Curvas  $n$ ,  $n^2$  e  $n^3$
- 3. Fibonacci: recursivo, memoizado e iterativo
- 4. Custo marginal  $\Delta T(n)$

```
using PlutoUI, Plots
@bind n Slider(1:100, default=20)
plot(ns, [ns, ns.^2, ns.^3])
```

# Contar, não cronometrar

- O aluno altera  $n$  e observa a diferença entre crescimento linear, quadrático e cúbico.
- A tabela é uma ponte entre código e função de custo.

```
ops_linear(n) = n  
ops_quadratico(n) = n^2  
ops_cubico(n) = n^3
```

```
[n ops_linear(n) ops_quadratico(n) ops_cubico(n)]
```

**ops\_linear, ops\_quadratico, ops\_cubico — para  $n = 20$  (valor padrão do slider)**

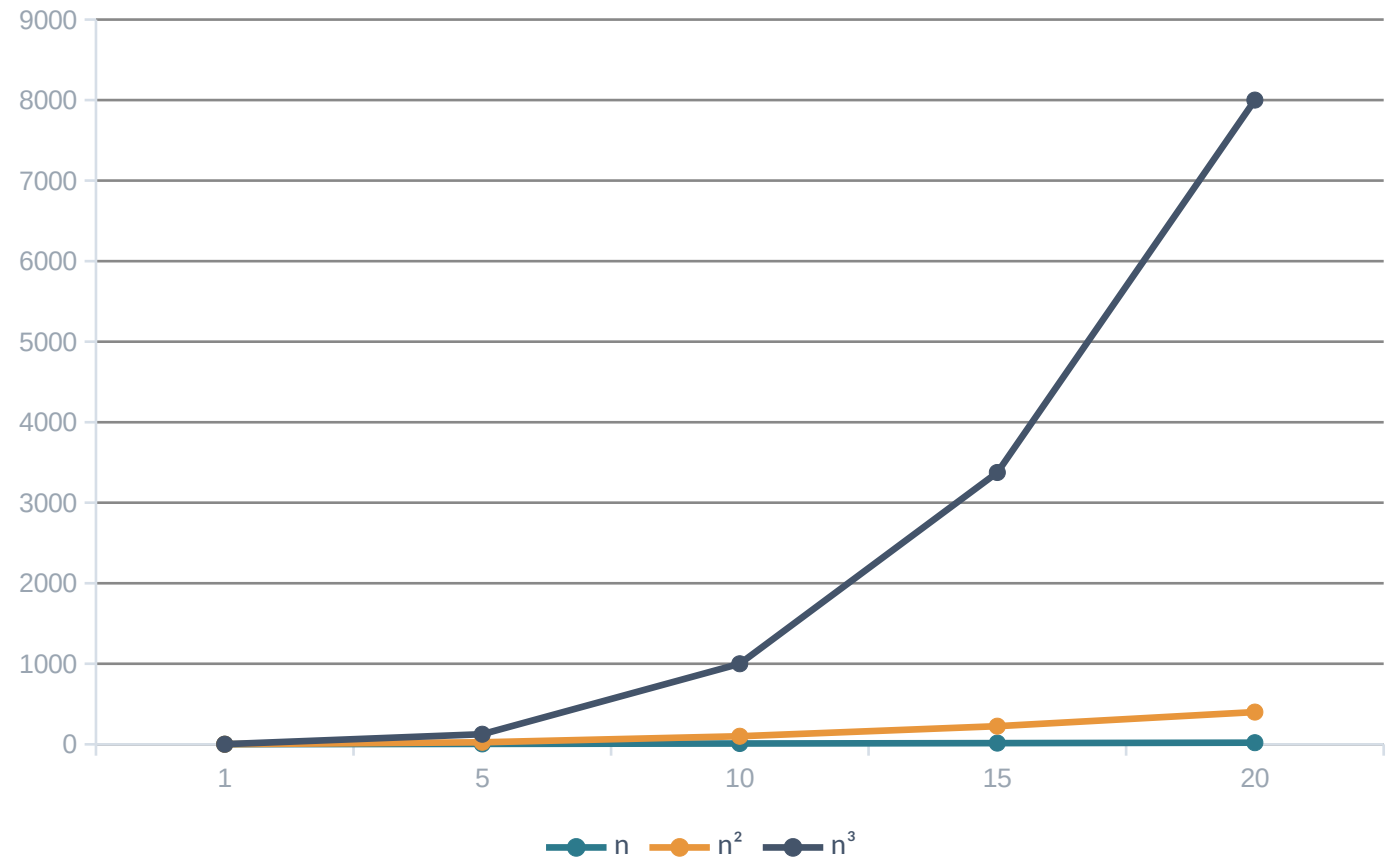
```
ops_linear(n) = n
ops_quadratico(n) = n^2
ops_cubico(n) = n^3

[n ops_linear(n) ops_quadratico(n) ops_cubico(n)]
```

```
n → 20 (valor padrão do slider)
ops_linear(20) → 20
ops_quadratico(20) →  $20^2 = 400$ 
ops_cubico(20) →  $20^3 = 8000$ 
resultado (vetor-linha) → [20 20 400 8000]
```

## Visualização

- A visualização torna perceptível a diferença entre “funcionar” e “escalar”.
- O gráfico é gerado a partir das funções de custo.



## Contagem de chamadas

- Para evitar travar a aula, o notebook modela a quantidade de chamadas pela recorrência de custo.
- O objetivo não é esperar o algoritmo terminar, mas entender por que ele explode.

```
calls_naive(n) =  
  n <= 1 ? 1 :  
  calls_naive(n-1) +  
  calls_naive(n-2) + 1  
  
fib_iter_ops(n) = max(n, 1)
```

## **calls\_naive(n) — quantas chamadas a recorrência prevê, para $n = 5$**

```
calls_naive(n) = n <= 1 ? 1 : calls_naive(n-1) + calls_naive(n-2) + 1
```

```
# chamada: calls_naive(5)
```

```
calls_naive(0) → 1 (caso-base)
```

```
calls_naive(1) → 1 (caso-base)
```

```
calls_naive(2) → calls_naive(1) + calls_naive(0) + 1 = 1 + 1 + 1 = 3
```

```
calls_naive(3) → calls_naive(2) + calls_naive(1) + 1 = 3 + 1 + 1 = 5
```

```
calls_naive(4) → calls_naive(3) + calls_naive(2) + 1 = 5 + 3 + 1 = 9
```

```
calls_naive(5) → calls_naive(4) + calls_naive(3) + 1 = 9 + 5 + 1 = 15 (bate com o total de chamadas contado na árvore de fib(5))
```

### $\Delta T(n)$

- A derivada discreta mostra o custo de aumentar a entrada em uma unidade.
- Essa é a ponte com coeficiente angular, inclinação e custo marginal.

```
delta(T, n) = T(n) - T(n-1)
```

```
linear(n) = 3n + 5
```

```
quadratica(n) = n^2
```

```
plot(ns[2:end], [delta.(Ref(linear), ns[2:end]),  
                 delta.(Ref(quadratica), ns[2:end])])
```

**delta(T, n) — para as funções linear e quadratica, em  $n = 4$**

```
delta(T, n) = T(n) - T(n-1)
```

```
linear(n) = 3n + 5
```

```
quadratica(n) = n^2
```

```
delta(linear, 4)
```

```
delta(quadratica, 4)
```

**delta(T, n)** → recebe uma função T como argumento e calcula  $T(n) - T(n-1)$

**linear(4)** →  $3 \times 4 + 5 = 17$

**linear(3)** →  $3 \times 3 + 5 = 14$

**delta(linear, 4)** →  $17 - 14 = 3$  (confere com  $\Delta T(n)=3$  da tabela “ $T(n)=3n+5$ ”)

**quadratica(4)** →  $4^2 = 16$

**quadratica(3)** →  $3^2 = 9$

**delta(quadratica, 4)** →  $16 - 9 = 7$  (confere com  $\Delta T(n)=7$  da linha  $n=4$  da tabela “ $T(n)=n^2$ ”)

## Atividade em sala

*Antes de executar, peça que os alunos respondam:*

- 1 Qual é a ação fundamental?
- 2 Quantas vezes ela ocorre?
- 3 A sequência gera constante aditiva ou multiplicativa?
- 4 O IF muda a ordem ou apenas o caminho?
- 5 Se dobrar  $n$ , o trabalho dobra, quadruplica ou cresce mais?

# Big O não é o ponto de partida

01

## Big O não é o ponto de partida

O ponto de partida é a função de custo  $T(n)$  do algoritmo — o Big O resume apenas o seu termo dominante.

02

## Laços moldam o crescimento

Sequências calibram constantes; laços aninhados multiplicam o crescimento — é aí que nascem  $O(n^2)$  e  $O(n \log n)$ .

03

## Programação dinâmica elimina recomputação

Guardar resultados de subproblemas troca uma explosão exponencial, como no Fibonacci ingênuo, por custo linear.



A SEGUIR — AULA 15 de 15

## Encerramento e Discussão

# Referências

RAABE, André; ZORZO, Avelino F.; BLIKSTEIN, Paulo. Computação na Educação Básica: Fundamentos e Experiências. Porto Alegre: Penso, 2020. Ebook. ISBN 9786581334048.

RIBEIRO, João Araújo. Introdução à Programação e aos algoritmos. Rio de Janeiro: LTC, 2019. ISBN 978-85-216-3640-3.